

Inter Process Communication(IPC)

Introduction

- An operating system (OS) allows multiple processes to run concurrently.
- Often, these processes need to exchange information or synchronize their actions.
- Inter-Process Communication (IPC) is the mechanism that enables processes to communicate and coordinate with each other.

For example:

- A web browser (one process) may communicate with a plug-in (another process).
- A client process may request data from a server process.

IPC mechanisms fall into two broad categories:

- Message Passing
- Shared Memory

a. Message Passing

- Processes communicate by sending and receiving messages.
- OS provides system calls for message transfer.
- No shared memory is required.

Advantages:

- Simpler to implement.
- Works for both local and remote processes.

Disadvantages:

- Message copying between processes can be slow.

b. Shared Memory

- A shared memory region is created in RAM.
- Multiple processes can read/write directly into this space.
- Synchronization (using semaphores/mutexes) is required to avoid race conditions.

Advantages:

- Faster since data is not copied.
- Efficient for large data exchange.

Disadvantages:

- Requires careful synchronization to avoid inconsistencies.

IPC Mechanisms in Operating Systems

Different operating systems (like UNIX, Linux, Windows) provide multiple IPC methods:

1. Pipes (Anonymous Pipes): Unidirectional communication channel between parent and child processes.
 - Example: Parent sends data to child via `write()`, child reads via `read()`.
2. Named Pipes (FIFOs): Like pipes but with a name in the file system. Can be used for unrelated processes.
 - Example: One process writes to FIFO, another reads.
3. Message Queues: OS-managed queues that store messages until a receiving process reads them. Support for priority messages.
 - Example: A server receives requests from multiple clients.

4. Shared Memory: Memory segment accessible by multiple processes. Fastest form of IPC since no copying is needed. Requires synchronization using semaphores.

5. Semaphores: Used for synchronization and preventing race conditions. Can be binary (mutex) or counting.

- Example: Producer-Consumer problem.

6. Sockets: Allow communication between processes, even across different machines (over a network).

- Example: Client-server communication using TCP/IP.

7. Signals: Lightweight form of IPC. A process can notify another process of an event (e.g., termination, alarm).

- Example: `kill -9 <pid>` sends a signal to terminate a process.

Processes

- Process Concept
- Process Scheduling
- Operations on Processes
- Cooperating Processes
- Inter process Communication
- Communication in Client-Server Systems

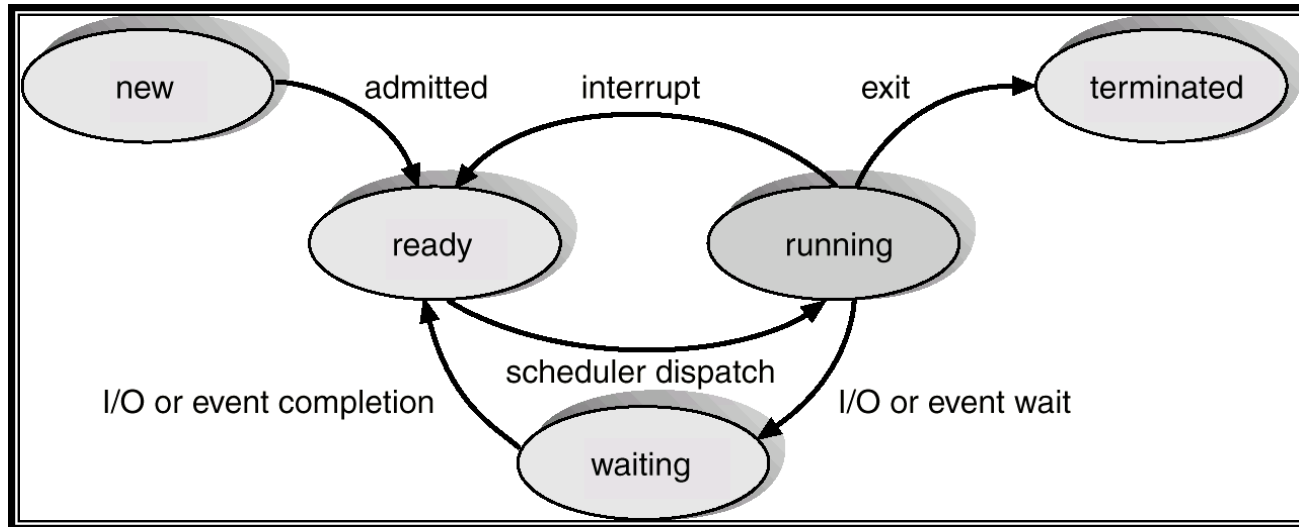
Process Concept

- ▶ An operating system executes a variety of programs:
 - Batch system – jobs
 - Time-shared systems – user programs or tasks
- ▶ Process – a program in execution; process execution must progress in sequential fashion.
- ▶ A process includes:
 - program counter
 - stack
 - data section

Process State

- ▶ As a process executes, it changes *state*
 - **new**: The process is being created.
 - **running**: Instructions are being executed.
 - **waiting**: The process is waiting for some event to occur.
 - **ready**: The process is waiting to be assigned to a processor.
 - **terminated**: The process has finished execution.

Diagram of Process State



Process Control Block (PCB)

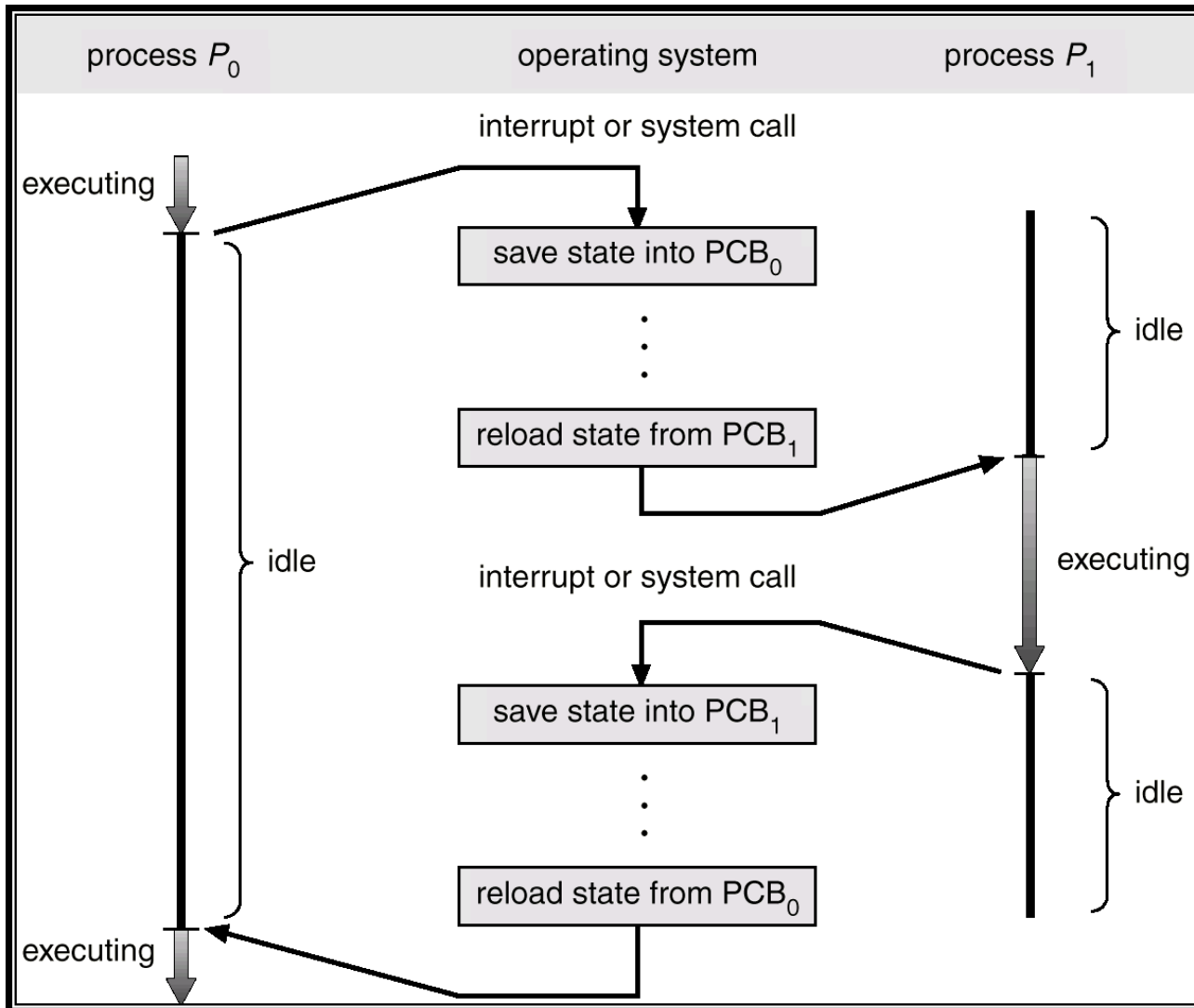
Information associated with each process.

- ▶ Process state
- ▶ Program counter
- ▶ CPU registers
- ▶ CPU scheduling information
- ▶ Memory-management information
- ▶ Accounting information
- ▶ I/O status information

Process Control Block (PCB)

pointer	process state
process number	
program counter	
registers	
memory limits	
list of open files	
• • •	

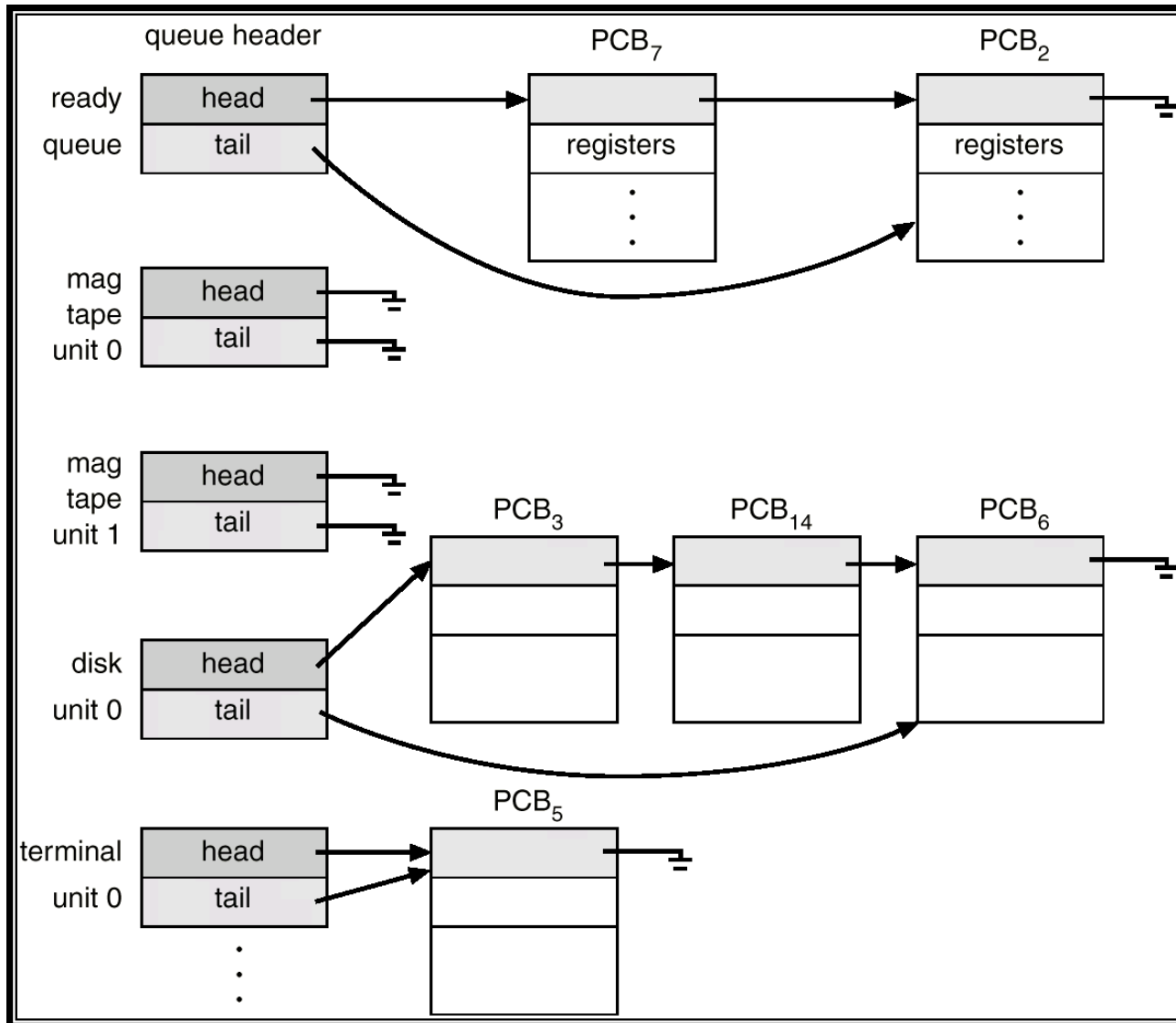
CPU Switch From Process to Process



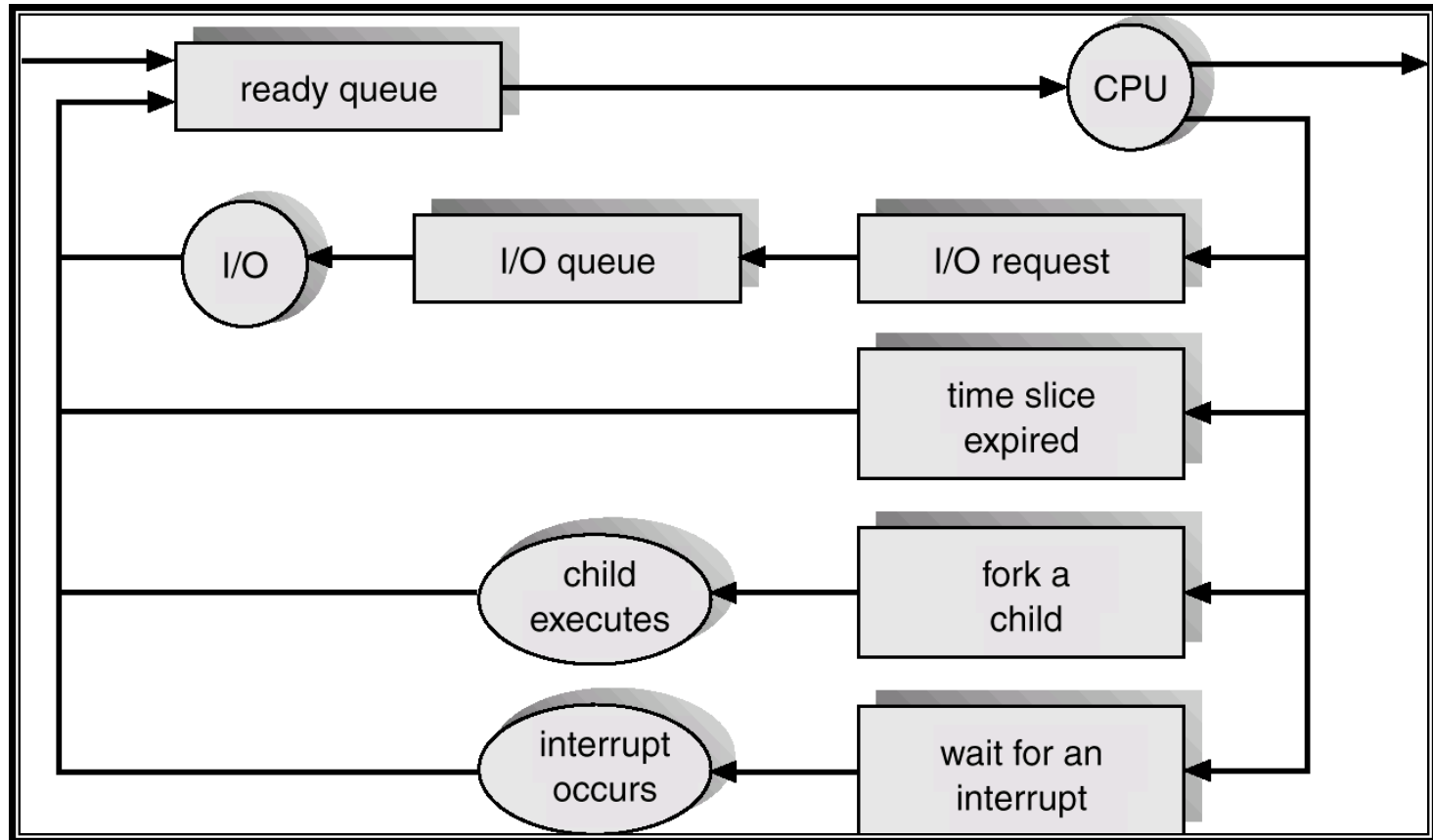
Process Scheduling Queues

- ▶ Job queue – set of all processes in the system.
- ▶ Ready queue – set of all processes residing in main memory, ready and waiting to execute.
- ▶ Device queues – set of processes waiting for an I/O device.
- ▶ Process migration between the various queues.

Ready Queue And Various I/O Device Queues



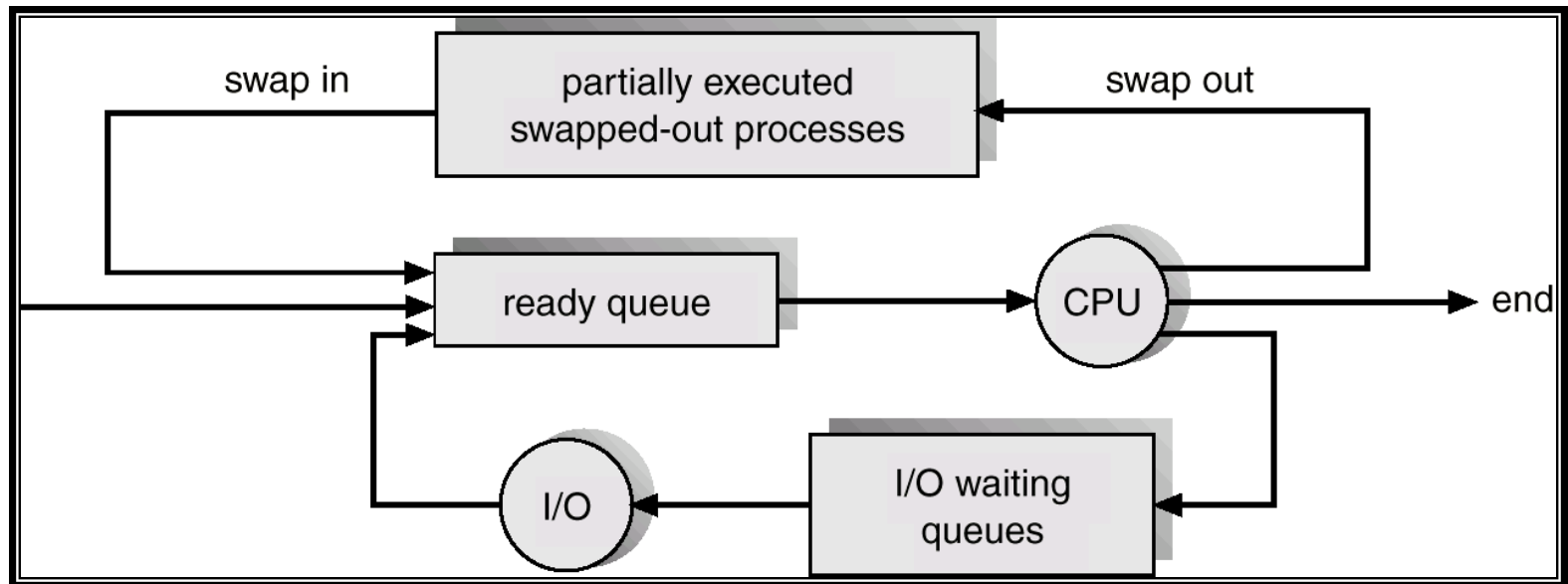
Representation of Process Scheduling



Schedulers

- ▶ Long-term scheduler (or job scheduler) – selects which processes should be brought into the ready queue.
- ▶ Short-term scheduler (or CPU scheduler) – selects which process should be executed next and allocates CPU.

Addition of Medium-Term Scheduling



Schedulers (Cont.)

- ▶ Short-term scheduler is invoked very frequently (milliseconds) $\Rightarrow$ (must be fast).
- ▶ Long-term scheduler is invoked very infrequently (seconds, minutes) $\Rightarrow$ (may be slow).
- ▶ The long-term scheduler controls the *degree of multiprogramming*.
- ▶ Processes can be described as either:
 - *I/O-bound process* – spends more time doing I/O than computations, many short CPU bursts.
 - *CPU-bound process* – spends more time doing computations; few very long CPU bursts.

Context Switch

- ▶ When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process.
- ▶ Context-switch time is overhead; the system does no useful work while switching.
- ▶ Time dependent on hardware support.

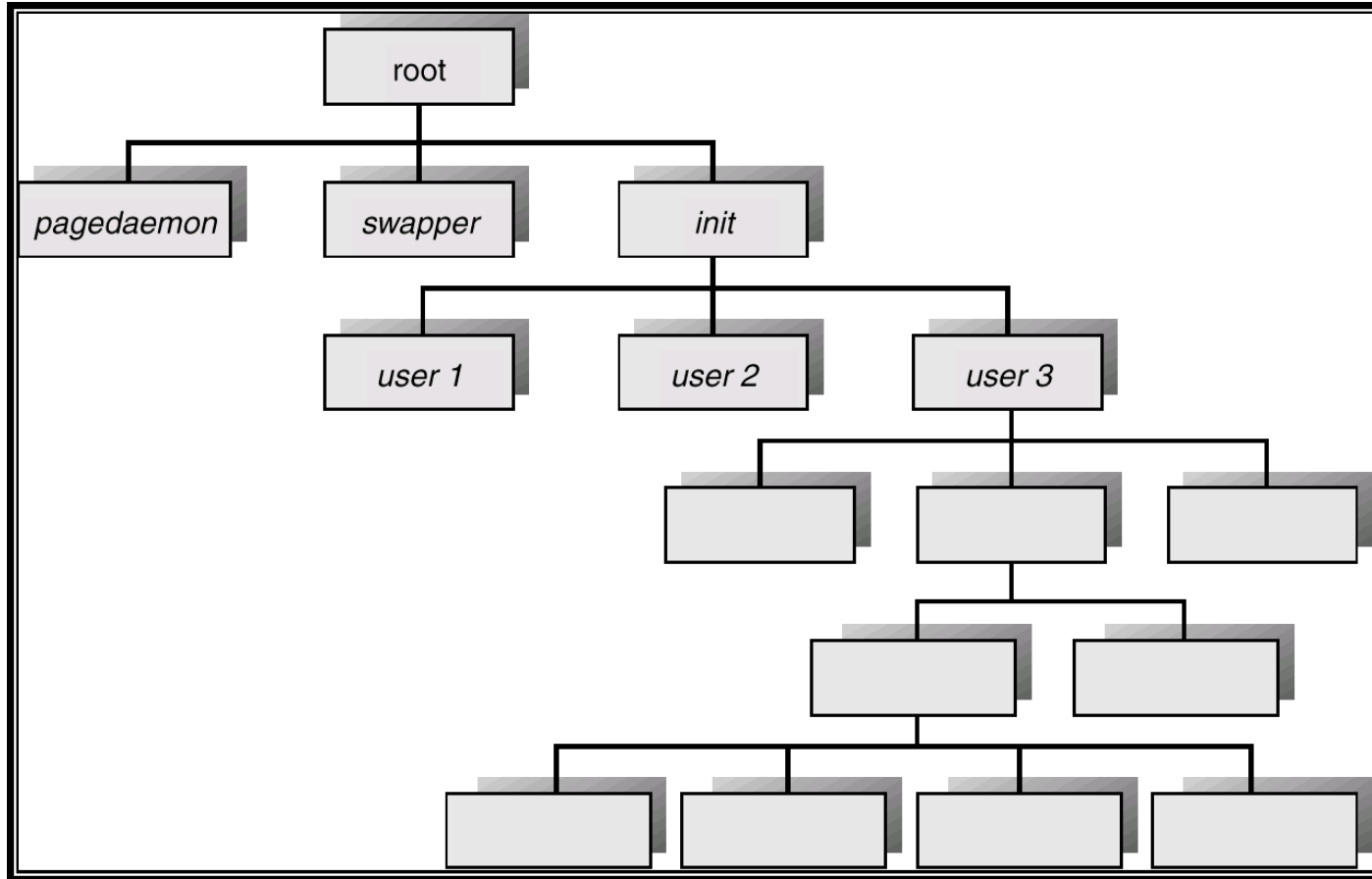
Process Creation

- Parent process create children processes, which, in turn create other processes, forming a tree of processes.
- Resource sharing
 - Parent and children share all resources.
 - Children share subset of parent's resources.
 - Parent and child share no resources.
- Execution
 - Parent and children execute concurrently.
 - Parent waits until children terminate.

Process Creation (Cont.)

- Address space
 - Child duplicate of parent.
 - Child has a program loaded into it.
- UNIX examples
 - **fork** system call creates new process
 - **exec** system call used after a **fork** to replace the process' memory space with a new program.

Processes Tree on a UNIX System



Process Termination

- ▶ Process executes last statement and asks the operating system to decide it (**exit**).
 - Output data from child to parent (via **wait**).
 - Process' resources are deallocated by operating system.
- ▶ Parent may terminate execution of children processes (**abort**).
 - Child has exceeded allocated resources.
 - Task assigned to child is no longer required.
 - Parent is exiting.
 - Operating system does not allow child to continue if its parent terminates.
 - Cascading termination.

Cooperating Processes

- *Independent* process cannot affect or be affected by the execution of another process.
- *Cooperating* process can affect or be affected by the execution of another process
- Advantages of process cooperation
 - Information sharing
 - Computation speed-up
 - Modularity
 - Convenience

Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process.
 - *unbounded-buffer* places no practical limit on the size of the buffer.
 - *bounded-buffer* assumes that there is a fixed buffer size.

Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use BUFFER_SIZE-1 elements

Bounded-Buffer – Producer Process

```
item nextProduced;
```

```
while (1) {  
    while (((in + 1) % BUFFER_SIZE) == out)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
}
```

Bounded-Buffer – Consumer Process

```
item nextConsumed;
```

```
while (1) {  
    while (in == out)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
}
```

Interprocess Communication (IPC)

- ▶ Mechanism for processes to communicate and to synchronize their actions.
- ▶ Message system – processes communicate with each other without resorting to shared variables.
- ▶ IPC facility provides two operations:
 - **send**(*message*) – message size fixed or variable
 - **receive**(*message*)
- ▶ If P and Q wish to communicate, they need to:
 - establish a *communication link* between them
 - exchange messages via send/receive
- ▶ Implementation of communication link
 - physical (e.g., shared memory, hardware bus)
 - logical (e.g., logical properties)

Interprocess Communications

- **Exchange of data** between two or more separate, independent **processes/threads**.
- **Operating systems** provide **facilities/resources** for inter-process communications (**IPC**), such as **message queues**, **semaphores**, and **shared memory**.
- **Distributed computing systems** make use of these **facilities/resources** to provide **application programming interface (API)** which allows IPC to be **programmed at a higher level** of abstraction. (e.g., **send** and **receive**)
- **Distributed computing** requires information to be **exchanged among independent processes**.

Indirect Communication

- Operations
 - create a new mailbox
 - send and receive messages through mailbox
 - destroy a mailbox
- Primitives are defined as:
 - send**(A , $message$) – send a message to mailbox A
 - receive**(A , $message$) – receive a message from mailbox A

Indirect Communication

- Mailbox sharing
 - P_1 , P_2 , and P_3 share mailbox A.
 - P_1 sends; P_2 and P_3 receive.
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes.
 - Allow only one process at a time to execute a receive operation.
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Synchronization

- Message passing may be either blocking or non-blocking.
- **Blocking** is considered **synchronous**
- **Non-blocking** is considered **asynchronous**
- **send** and **receive** primitives may be either blocking or non-blocking.

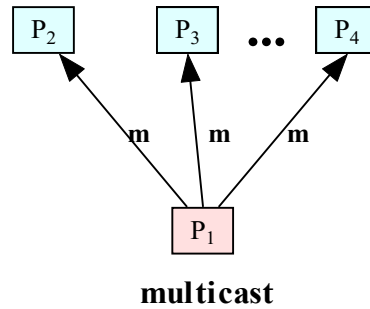
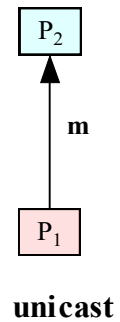
Buffering

- Queue of messages attached to the link; implemented in one of three ways.
 1. Zero capacity – 0 messages
Sender must wait for receiver (rendezvous).
 2. Bounded capacity – finite length of n messages
Sender must wait if link full.
 3. Unbounded capacity – infinite length
Sender never waits.

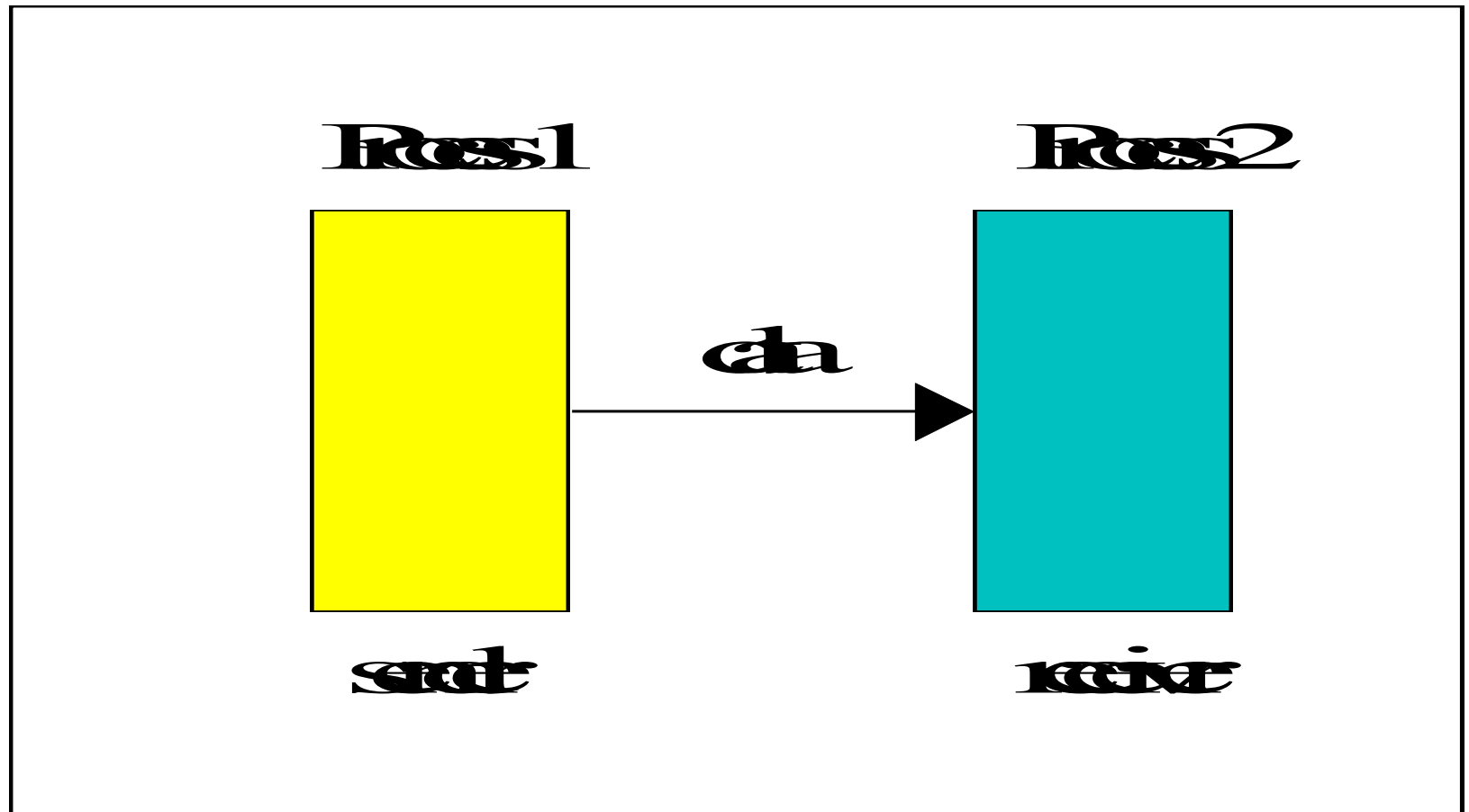
IPC – unicast and multicast

- In distributed computing, two or more processes engage in **IPC** using a **protocol agreed upon** by the **processes**. A process may be a **sender** at some points during a protocol, a **receiver** at other points.
- When communication is from **one process** to a **single** other **process**, the IPC is said to be a **unicast**, e.g., **Socket communication**. When communication is from **one process** to a **group** of **processes**, the IPC is said to be a **multicast**, e.g.,

Unicast vs. Multicast



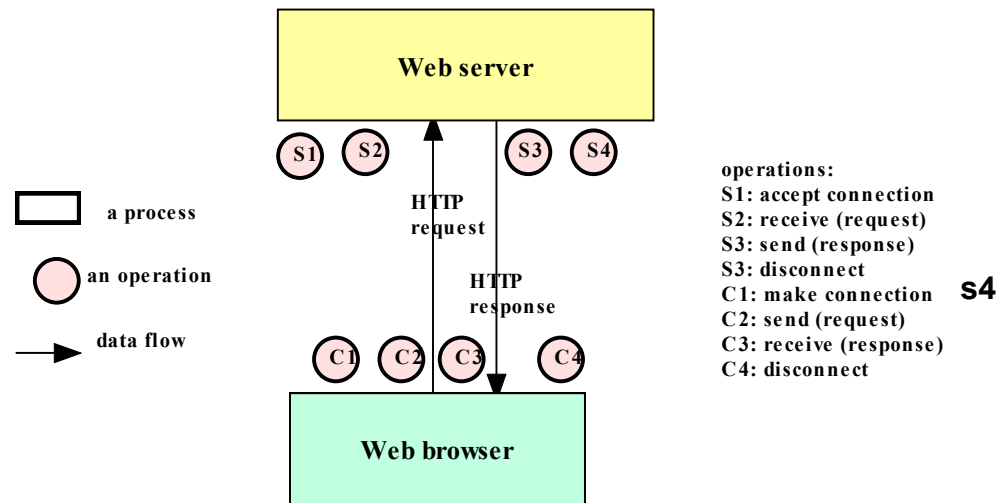
Interprocess Communications in Distributed Computing



Operations provided in an archetypal Interprocess Communications API

- **Receive** ([sender], message storage object)
- **Connect** (sender address, receiver address), for connection-oriented communication.
- **Send** ([receiver], message)
- **Disconnect** (connection identifier), for connection-oriented communication.

Interprocess Communication in basic HTTP



Processing order: C1, S1, C2, S2, S3, C3, C4, S4

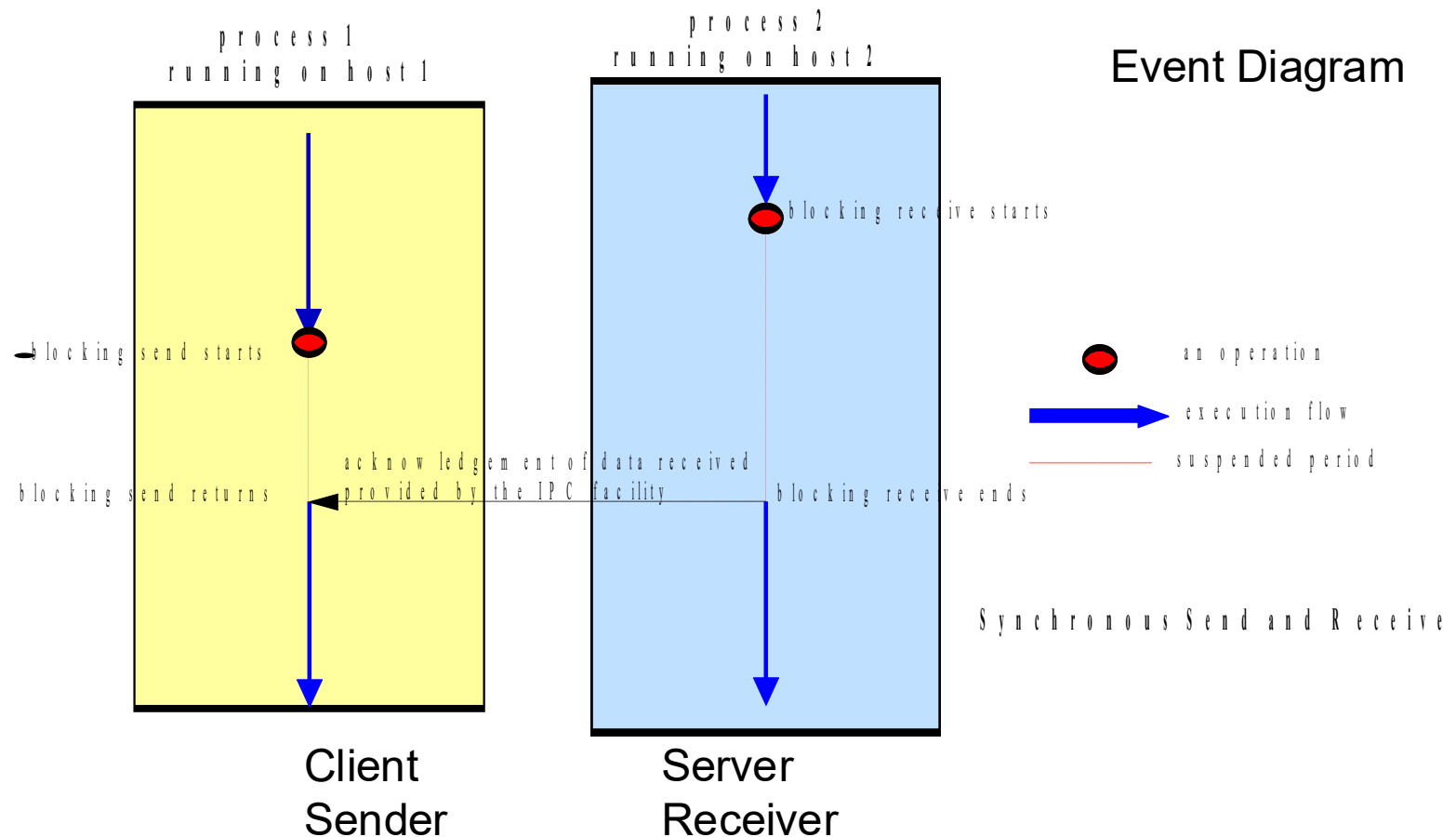
Event Synchronization

- Interprocess communication may require that the **two processes** **synchronize** their **operations**: one side sends, then the other receives until all data has been sent and received.
- Ideally, the **send operation starts** before the receive operation commences.
- In practice, the **synchronization** requires system support.

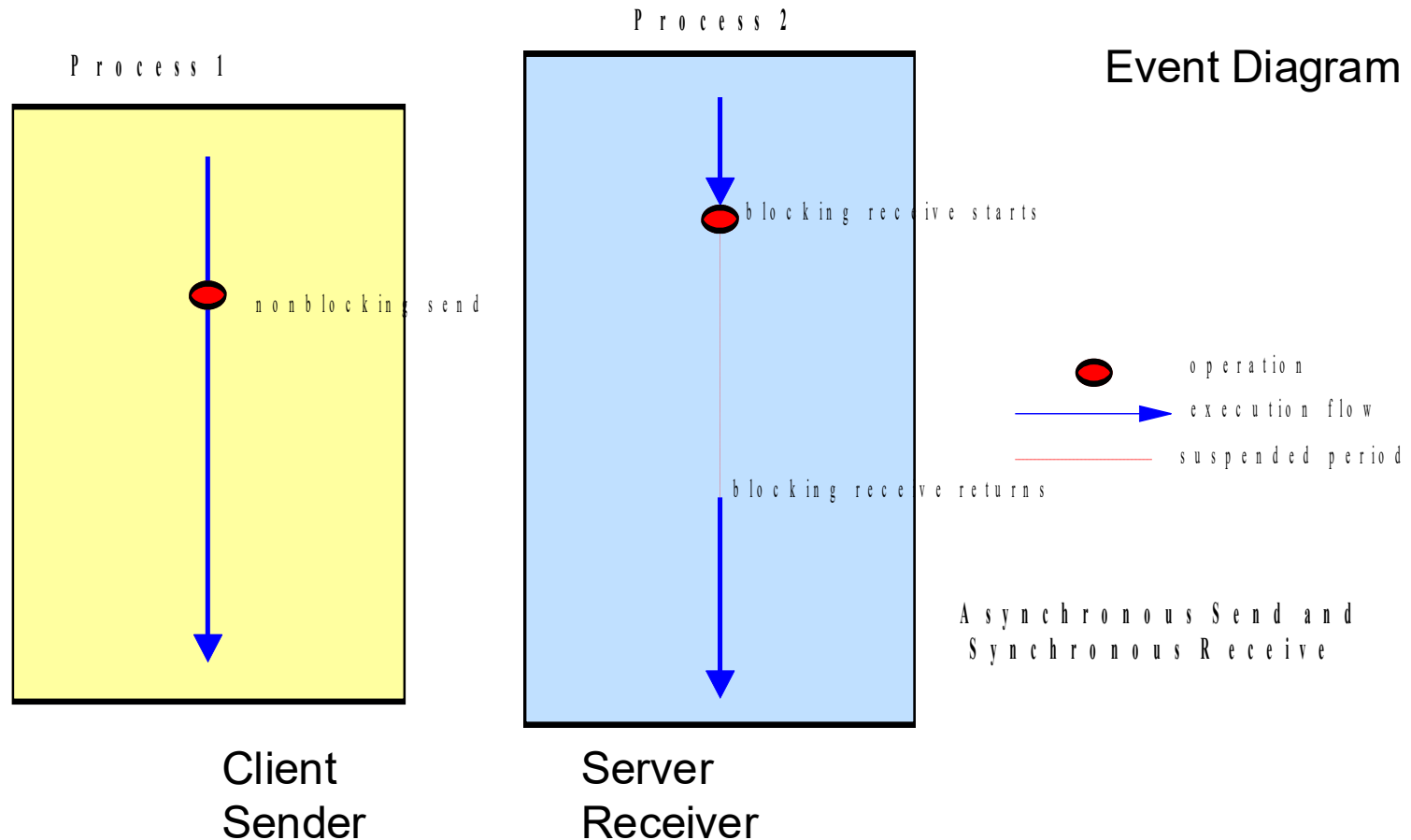
Synchronous vs. Asynchronous Communication

- ▶ The IPC operations may provide the **synchronization** necessary using **blocking**. A **blocking operation** issued by a process will block further processing of the process until the operation is fulfilled.
- ▶ Alternatively, **IPC** operations may be **asynchronous** or **nonblocking**. An **asynchronous operation** issued by a process will not block further processing of the process. Instead, the process is **free to proceed** with its processing, and may optionally be notified by the system when the operation is fulfilled.

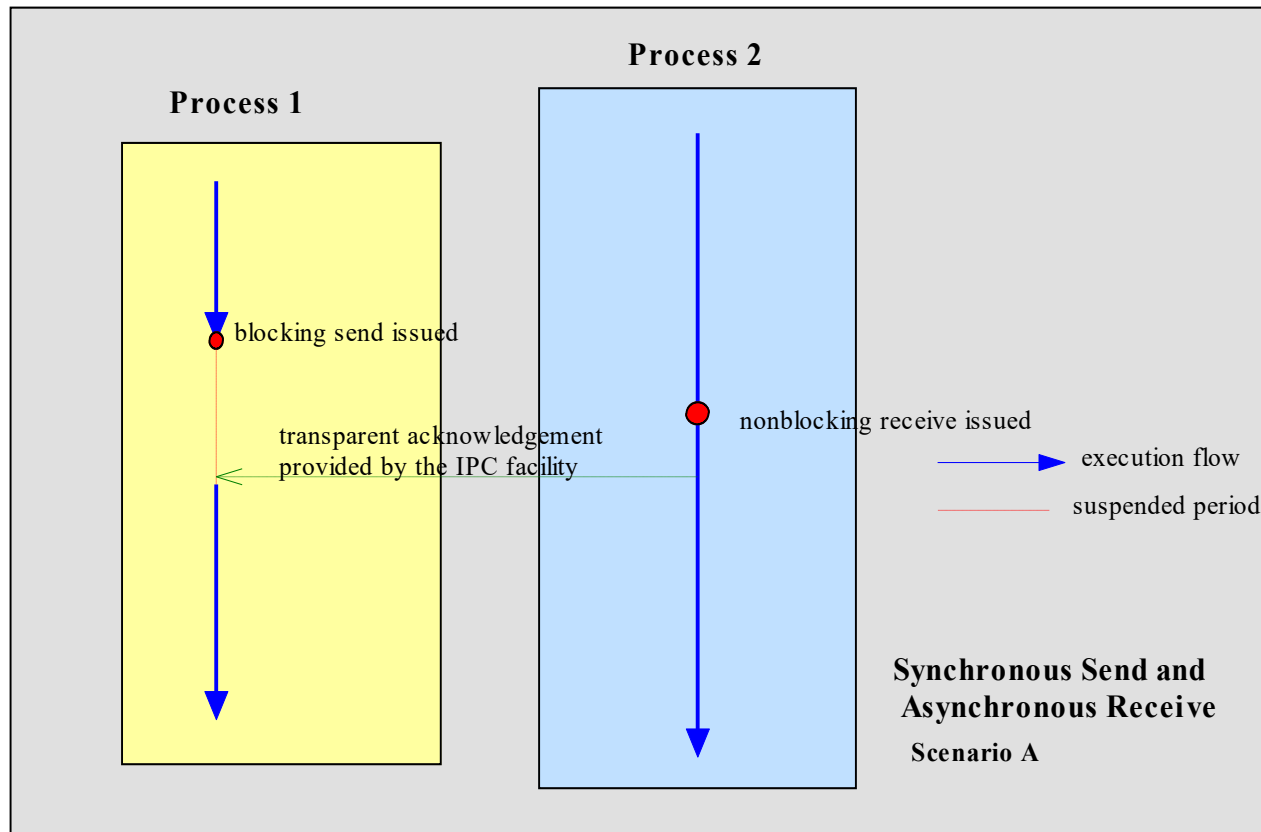
Synchronous send and receive



Asynchronous send and synchronous receive

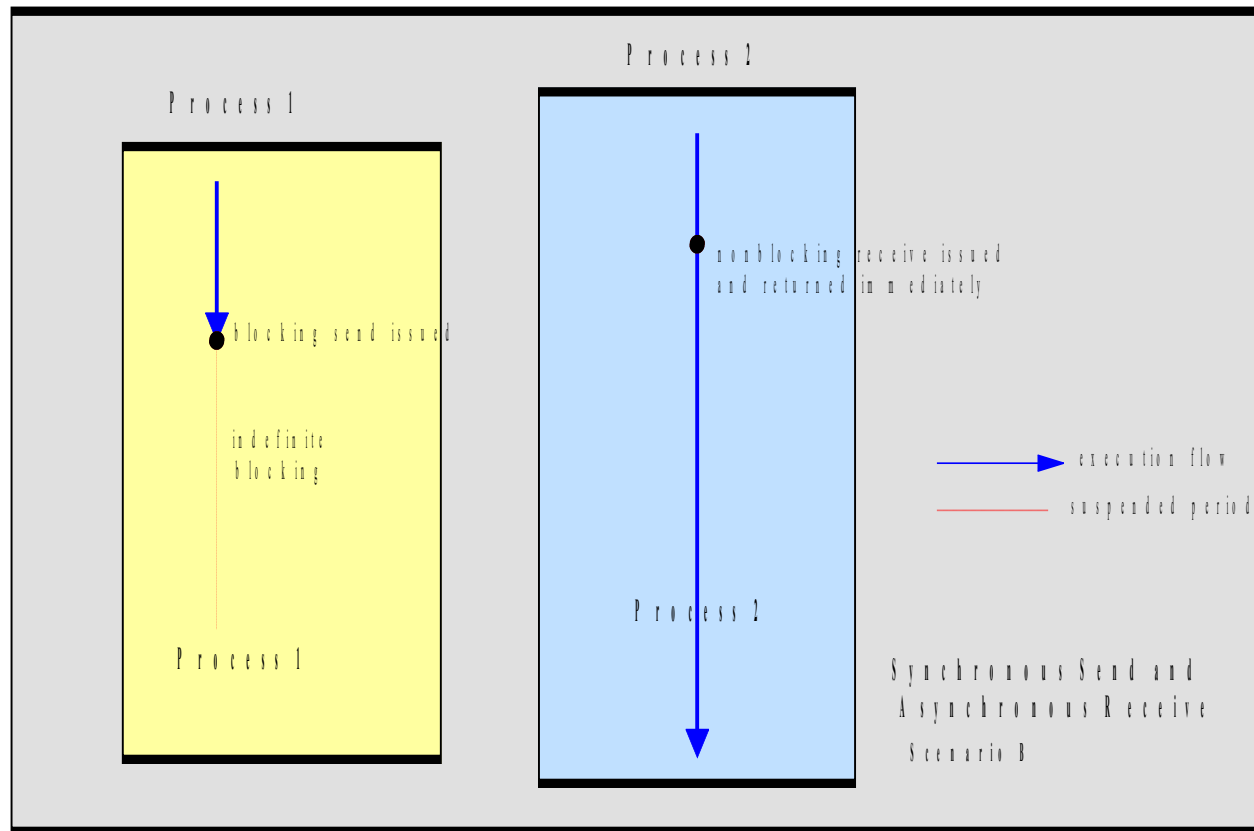


Synchronous send and Async. Receive - 1



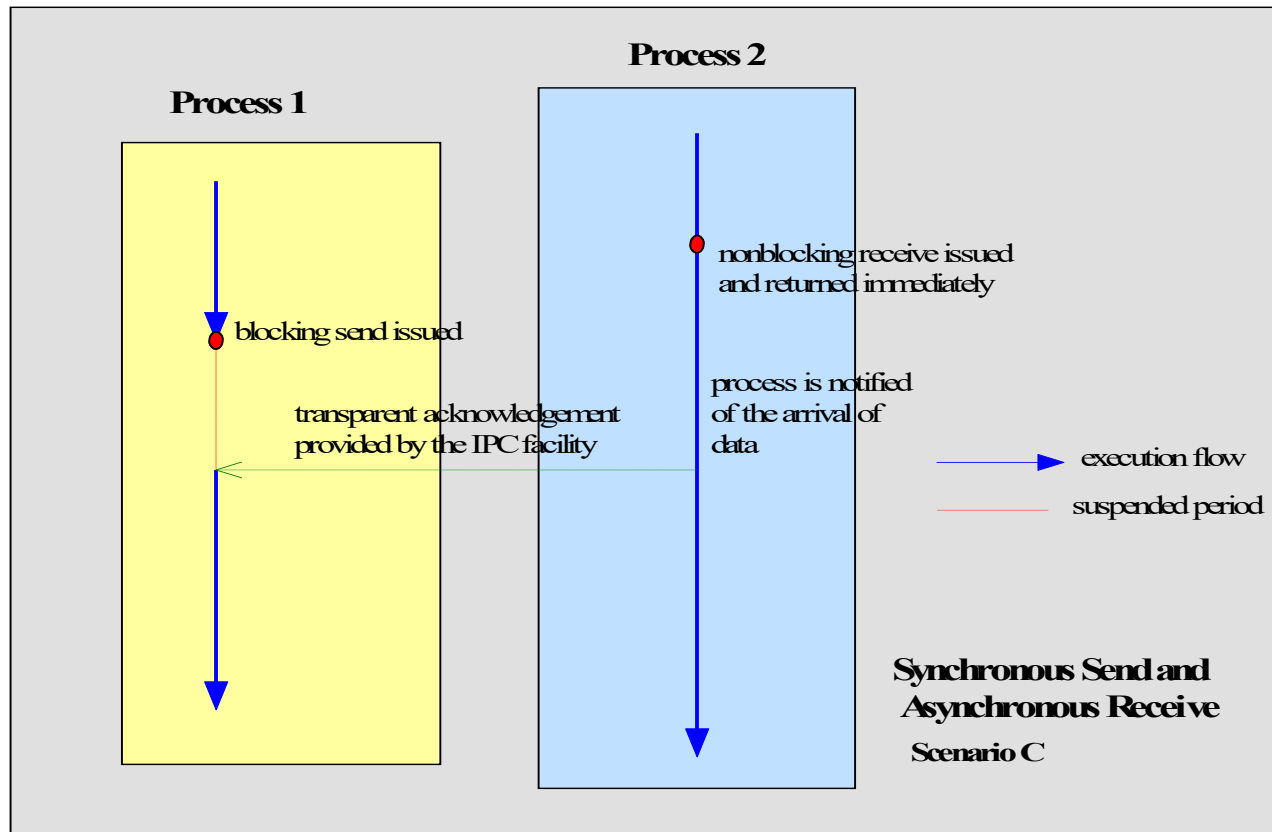
Data from P1 was received by P2
before issuing a non-blocking receive op in P2

Synchronous send and Async. Receive - 2



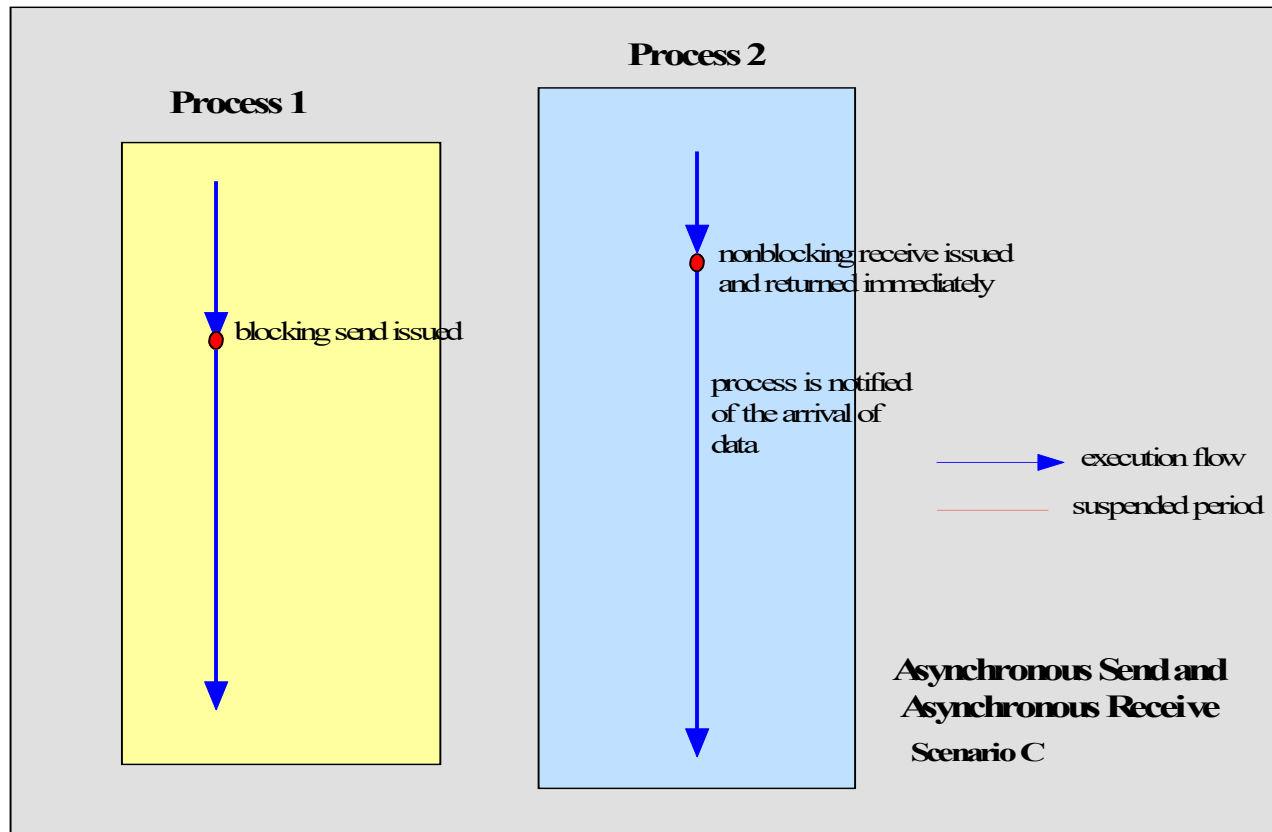
Data from P1 arrived to P2 **after** P2 issued a non-blocking receive op

Synchronous send and Async. Receive - 3



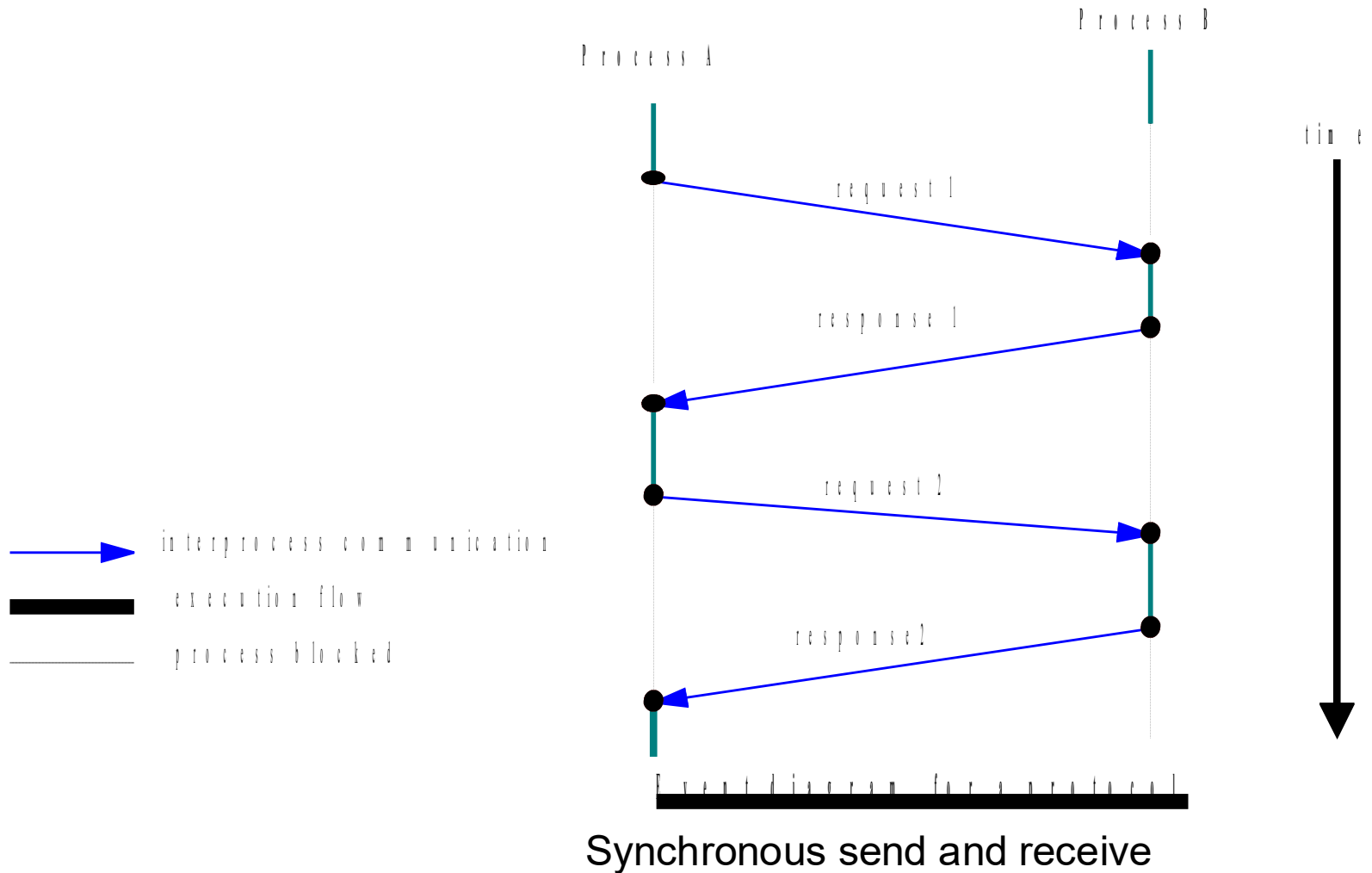
Data from P1 arrived to P2 **before** P2 issues a non-blocking receive op.
P2 is notified of the arrival of data

Asynchronous send and Asynchronous receive



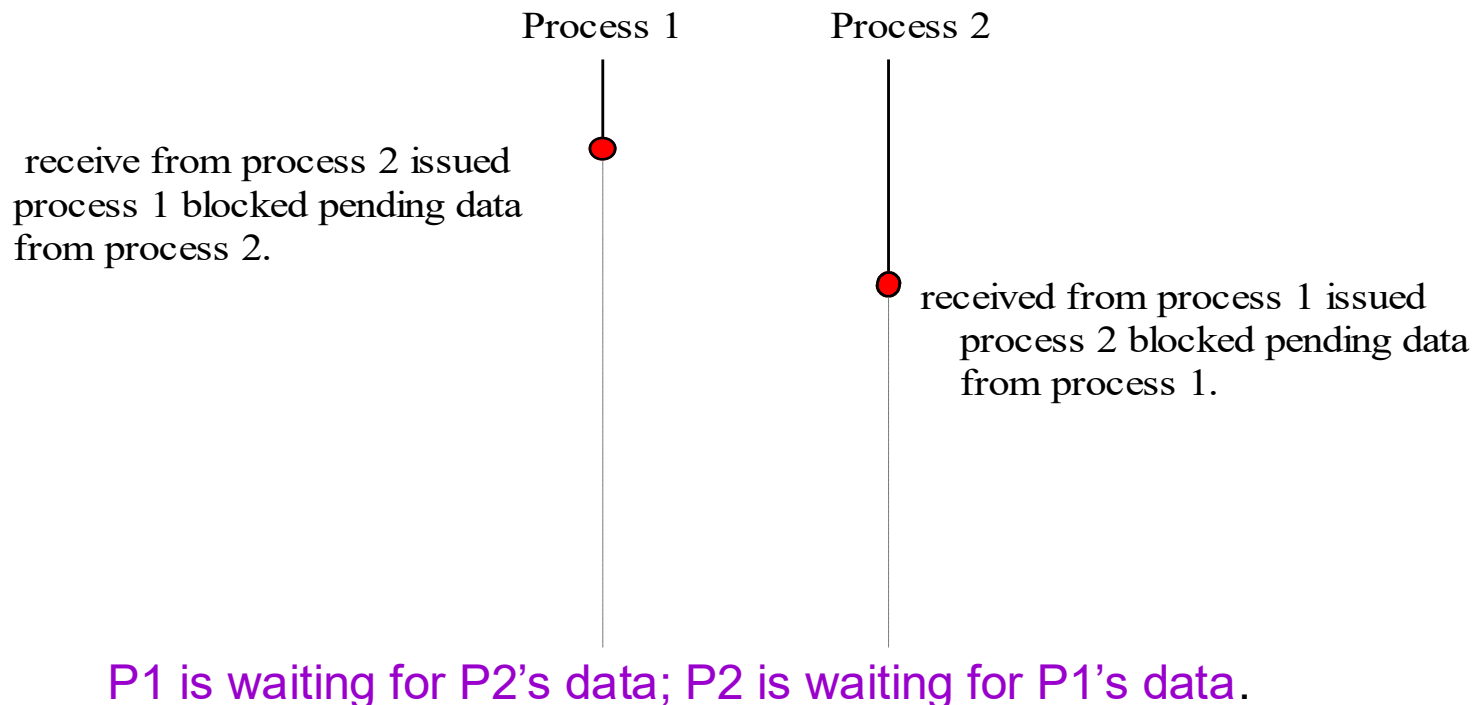
Does P1 need an acknowledgement from P2?

Event diagram



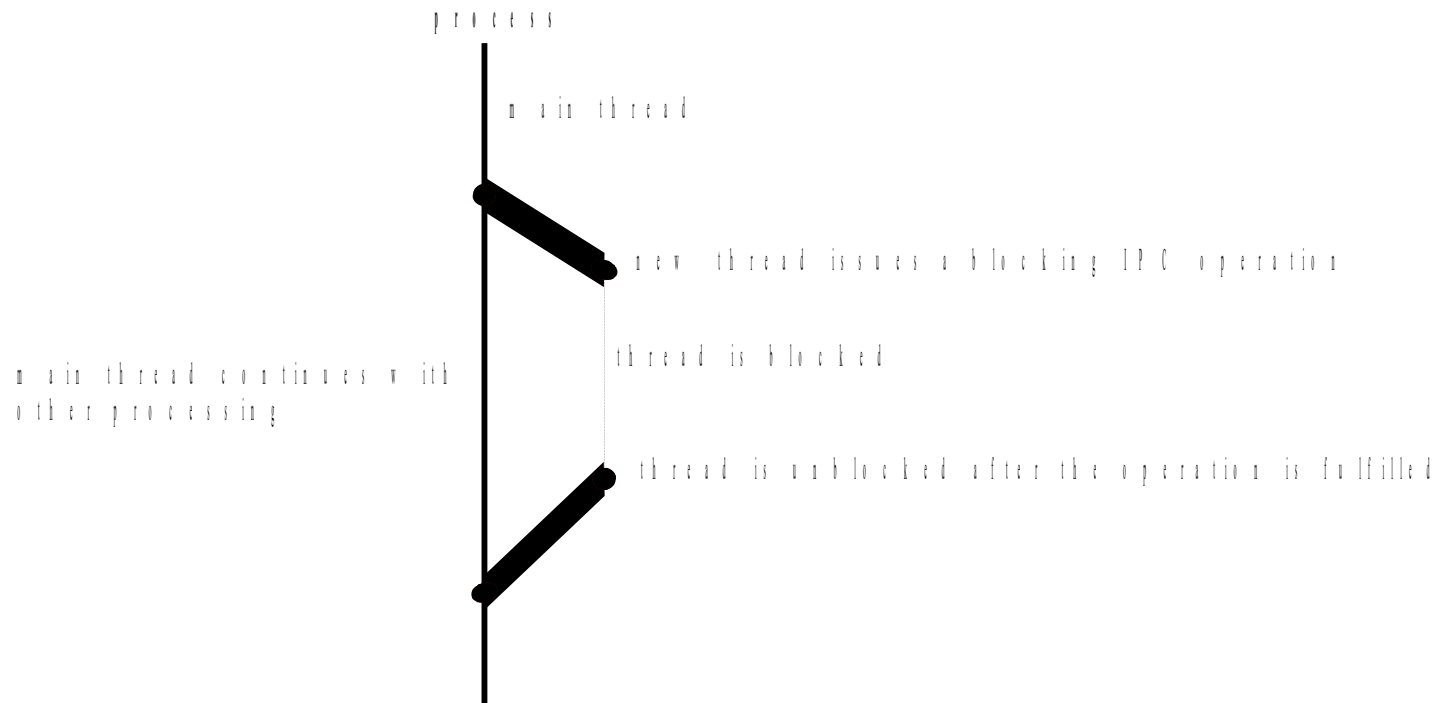
Blocking, deadlock, and timeouts

- **Blocking** operations issued in the **wrong sequence** can cause **deadlocks**.
- Deadlocks should be avoided. Alternatively, **timeout** can be used to detect deadlocks.



Using threads for asynchronous IPC

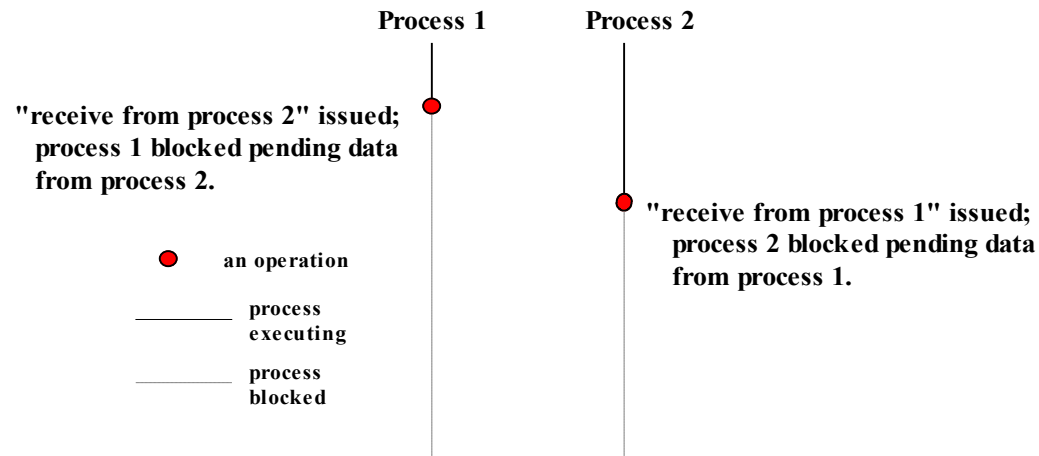
- When using an IPC programming interface, it is important to note whether the **operations are synchronous or asynchronous**.
- If only **blocking operation** is provided for send and/or receive, then it is the programmer's responsibility to using **child processes** or **threads** if **asynchronous operations** are desired.



Deadlocks and Timeouts

- ▶ **Connect** and **receive** operations can result in **indefinite blocking**
- ▶ For example, a **blocking connect request** can result in the requesting process to be suspended indefinitely if the connection is unfulfilled or cannot be fulfilled, perhaps as a result of a breakdown in the network .
- ▶ It is generally unacceptable for a requesting process to “hang” indefinitely. **Indefinite blocking** can be avoided by using **timeout**.
- ▶ Indefinite blocking may also be caused by a **deadlock**

Indefinite blocking due to a deadlock



P1 is waiting for P2's data; P2 is waiting for P1's data.

Data Representation

- ▶ **Data** transmitted on the network is a **binary stream**.
- ▶ An interprocess communication system may provide the capability to allow **data representation** to be imposed on the **raw** data.
- ▶ Because different computers may have **different internal storage format** for the **same data type**, an **external representation** of data may be necessary—**standard format**.
- ▶ **Data marshalling** is the process of (i) flattening a data structure, and (ii) converting the data to an **external representation**.
- ▶ Some well known **external data representation schemes** are:
 - Sun **XDR** (External Data Representation)
 - ASN.1** (Abstract Syntax Notation One)
 - XML** (Extensible Markup Language)

Direct Communication

- Processes must name each other explicitly:
 - **send** (*P, message*) – send a message to process P
 - **receive**(*Q, message*) – receive a message from process Q
- Properties of communication link
 - Links are established automatically.
 - A link is associated with exactly one pair of communicating processes.
 - Between each pair there exists exactly one link.
 - The link may be unidirectional, but is usually bi-directional.

Indirect Communication

- ▶ Messages are directed and received from mailboxes (also referred to as ports).
 - Each mailbox has a unique id.
 - Processes can communicate only if they share a mailbox.
- ▶ Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes.
 - Each pair of processes may share several communication links.
 - Link may be unidirectional or bi-directional.

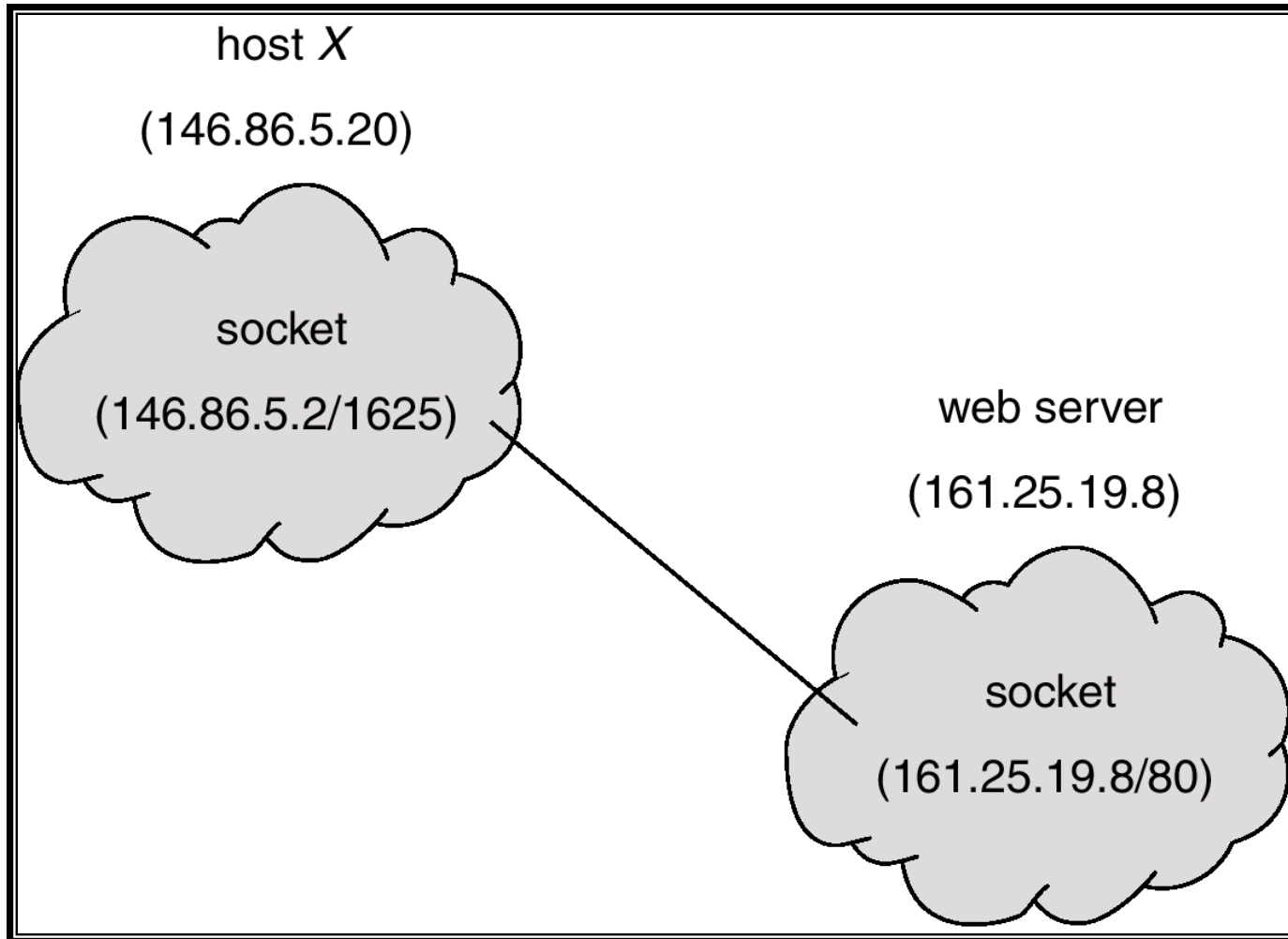
Client-Server Communication

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)

Sockets

- A socket is defined as an *endpoint for communication*.
- Concatenation of IP address and port
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets.

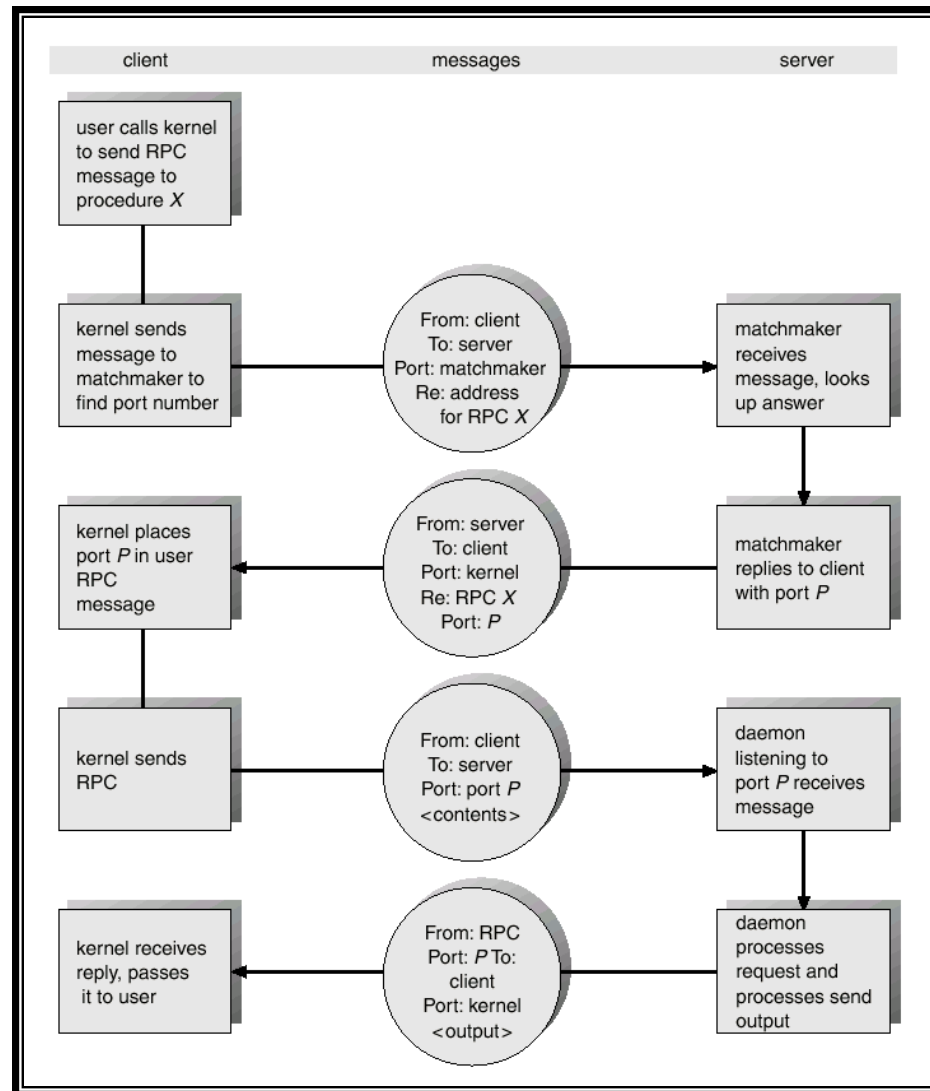
Socket Communication



Remote Procedure Calls

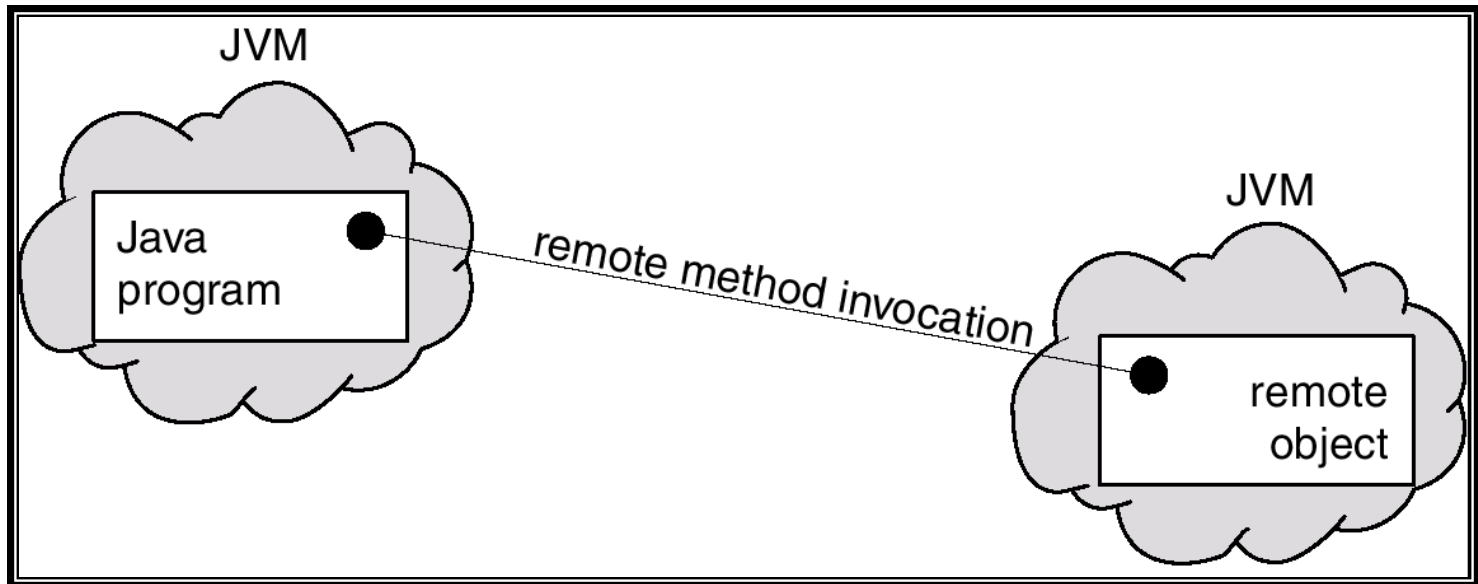
- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems.
- **Stubs** – client-side proxy for the actual procedure on the server.
- The client-side stub locates the server and *marshalls* the parameters.
- The server-side stub receives this message, unpacks the marshalled parameters, and performs the procedure on the server.

Execution of RPC

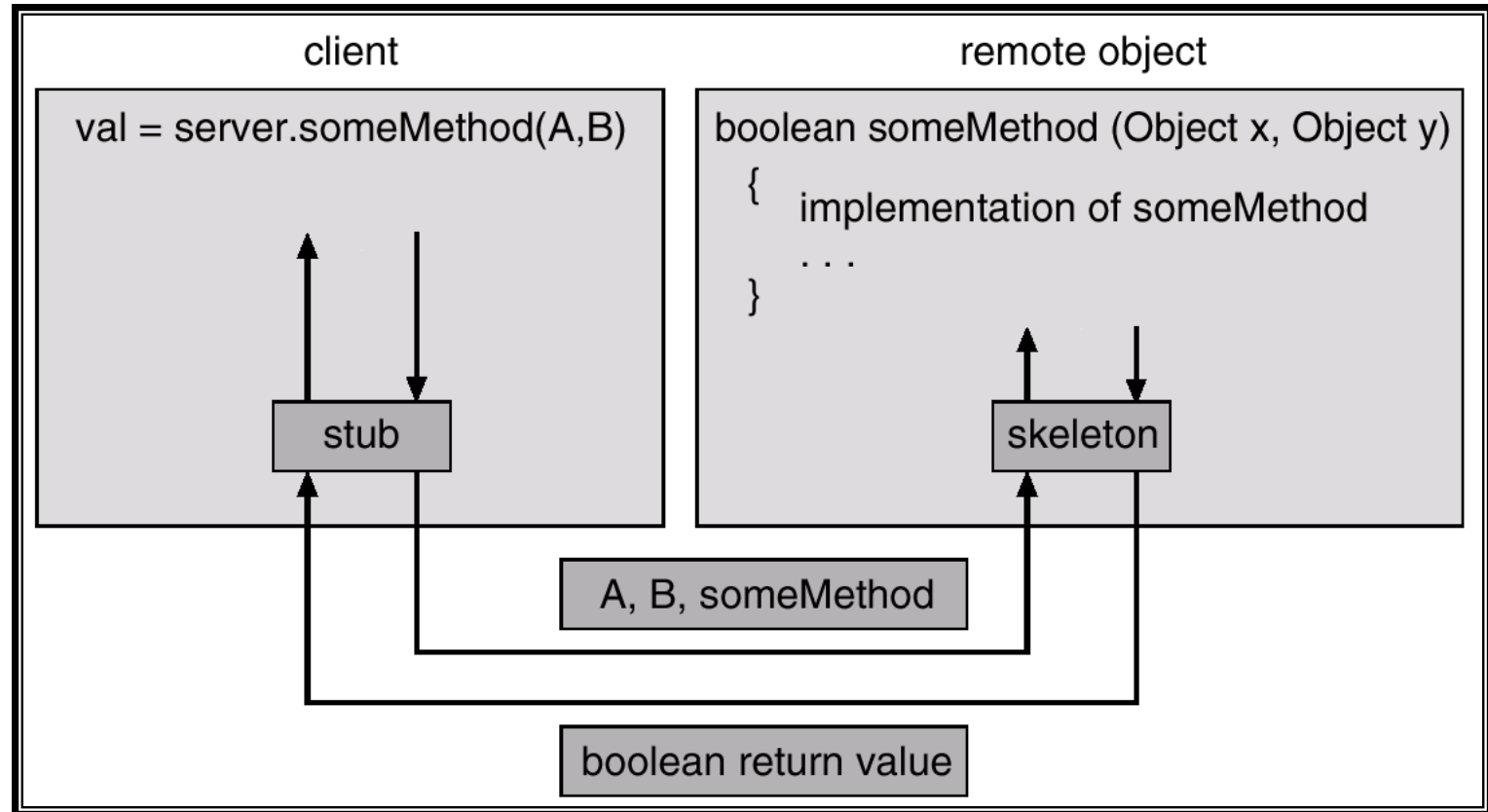


Remote Method Invocation

- Remote Method Invocation (RMI) is a Java mechanism similar to RPCs.
- RMI allows a Java program on one machine to invoke a method on a remote object.



Marshalling Parameters



Data marshalling is the process of (i) flattening a data structure, and (ii) converting the data to an **external representation**.

IPC paradigms and implementations

Paradigms of IPC of different levels of abstraction have evolved, with corresponding implementations.

**level of
abstraction**

IPC paradigms

Example IPC Implementations



remote procedure/method

Remote Procedure Call (RPC), Java RMI

socket API

Unix socket API, Winsock

data transmission

serial/parallel communication

UNIX Socket:

<http://www.ecst.csuchico.edu/~beej/guide/ipc/usock.html>